Diseño de patrones y seguridad

MediConecta · Desarrollo de Aplicaciones II · TP Integrador
Grupo 4: Casanova Sergio, Bodas Agustín, Bonarrico Máximo

1. Alcance

Este documento describe los patrones de diseño aplicados en MediConecta y el diseño de seguridad del sistema. Para cada patrón se indica el problema concreto del dominio que resuelve, las clases que cumplen cada rol, un fragmento de código de referencia y las consecuencias de haberlo elegido frente a la alternativa descartada. Los patrones se agrupan en cuatro familias: Java EE (Alur), GoF, integración y mensajería (Hohpe y Woolf) y resiliencia.

Se cierra con los patrones que se evaluaron y se descartaron, y con la trazabilidad de cada ítem del checklist de la cátedra.

2. Mapa de patrones

La columna Entrega indica cuándo se incorpora cada patrón. Base significa que pertenece a la estructura en capas de la Entrega 1.

#	**Patrón**	**Familia**	**Clase o componente**	**Entrega**
1	Data Access Object	Java EE	TurnoDAO, PacienteDAO, ProfesionalDAO	Base
2	Transfer Object (DTO)	Java EE	TurnoDTO, SolicitudTurnoDTO, FichaPacienteDTO	Base
3	Session Facade	Java EE	TurnoFacade (@Stateless)	Base
4	Singleton	GoF / EJB	CatalogoPrestaciones (@Singleton)	Base
5	Builder	GoF	EventoTurno.Builder	E3 (19/10)
6	Template Method + Factory Method	GoF	ProcesadorNotificacion y subclases	E3 (19/10)
7	Transactional Client	Integración	TurnoService: envío JMS dentro de la transacción JTA	E3 (19/10)
8	Publish-Subscribe Channel	Integración	topico.turnos	E3 (19/10)
9	Point-to-Point Channel	Integración	cola.autorizaciones, cola.revalidacion	E3 (19/10)
10	Event-Driven Consumer	Integración	Message Driven Beans	E3 (19/10)
11	Competing Consumers	Integración	AutorizacionConsumer (dos instancias)	E3 (19/10)
12	Selective Consumer	Integración	Selector sobre la propiedad tipoEvento	E3 (19/10)
13	Durable Subscriber	Integración	Suscripciones de Facturación y Auditoría	E3 (19/10)
14	Idempotent Receiver	Integración	Tabla evento_procesado con clave única	E3 (19/10)
15	Dead Letter Channel y Message Expiration	Integración	address-setting de WildFly y TTL del productor	E3 (19/10)
16	Adapter / Gateway	GoF / Fowler	CoberturaGateway, CoberturaSoapAdapter	E4 (09/11)
17	Circuit Breaker, Retry y Fallback con Cache-Aside	Resiliencia	CoberturaSoapAdapter con MicroProfile Fault Tolerance	E4 (09/11)
18	Chain of Responsibility	GoF	ValidadorReserva y sus eslabones	E4 (09/11)
19	State	GoF	EstadoTurno y sus estados	E4 (09/11)
20	Strategy	GoF	CalculoCopago (y PoliticaListaEspera en la final)	E4 (09/11)
21	Intercepting Filter	Java EE	AuditoriaInterceptor (interceptor CDI)	E4 (09/11)
22	Decorator	GoF / CDI	PacienteEnmascarado (@Decorator)	Final (30/11)

3. Patrones Java EE

3.1 Data Access Object y Transfer Object

Aspecto	Detalle
Problema en MediConecta	La lógica de turnos no debe depender de cómo se consulta la base, y las entidades JPA no deben salir de la aplicación: exponerlas en REST o SOAP filtra campos internos (como el diagnóstico) y acopla el contrato público al modelo de datos.
Solución	Cada agregado tiene su DAO con las consultas JPA parametrizadas. Hacia afuera viajan DTO armados por un mapper, con solo los campos que el consumidor necesita.
Participantes (rol del patrón: clase)	DAO: TurnoDAO, PacienteDAO. Transfer Object: TurnoDTO, FichaPacienteDTO. Assembler: TurnoMapper.
Consecuencias y alternativa descartada	Cambiar el esquema de la base no rompe la API. Costo: duplicar estructuras (entidad y DTO). Se descartó exponer las entidades directamente por el riesgo de filtrar datos clínicos.

3.2 Session Facade

Aspecto	Detalle
Problema en MediConecta	El cliente REST tendría que coordinar validación, persistencia, publicación del evento y cálculo de copago, y cada paso abriría su propia transacción.
Solución	Un EJB stateless expone las operaciones de negocio completas. Es la frontera transaccional (atributo REQUIRED por defecto) y el punto donde se aplica la seguridad declarativa.
Participantes (rol del patrón: clase)	Facade: TurnoFacade. Subsistemas: TurnoService, ValidadorReserva, CoberturaGateway, CalculoCopago.
Consecuencias y alternativa descartada	Una operación de negocio es una transacción y un único punto de control de acceso. Se descartó Business Delegate porque no hay clientes remotos de EJB: el acceso externo es por REST y SOAP.

```
@Stateless
@DeclareRoles({"PACIENTE", "PROFESIONAL", "ADMINISTRATIVO", "AUDITOR", "SISTEMA"})
public class TurnoFacade {
    @Inject TurnoService turnos;

    @RolesAllowed({"PACIENTE", "ADMINISTRATIVO"})
    public TurnoDTO reservar(SolicitudTurnoDTO solicitud) { return turnos.reservar(solicitud); }

    @RolesAllowed({"PACIENTE", "PROFESIONAL", "ADMINISTRATIVO"})
    public void cancelar(long idTurno) { turnos.cancelar(idTurno); }
}
```

3.3 Intercepting Filter (interceptor CDI de auditoría)

Aspecto	Detalle
Problema en MediConecta	Los datos de salud exigen registrar quién accedió a qué y cuándo. Escribir ese registro dentro de cada método lo repite en todo el código y es fácil olvidarlo.
Solución	Una anotación @Auditado y un interceptor que envuelve la invocación, registra usuario, operación, recurso y resultado, y deja pasar la llamada.
Participantes (rol del patrón: clase)	Filtro: AuditoriaInterceptor. Destino: los métodos anotados con @Auditado. Registro: RegistroAuditoria.
Consecuencias y alternativa descartada	Agregar auditoría a un método es una anotación. El registro guarda identificadores, nunca el dato clínico en sí. Se descartó un filtro HTTP porque no cubre las invocaciones que llegan por SOAP ni por mensajería.

```
@InterceptorBinding @Retention(RUNTIME) @Target({TYPE, METHOD})
public @interface Auditado {}

@Auditado @Interceptor @Priority(Interceptor.Priority.APPLICATION)
public class AuditoriaInterceptor {
    @Inject SecurityContext seguridad;
    @Inject RegistroAuditoria registro;

    @AroundInvoke
    public Object auditar(InvocationContext ctx) throws Exception {
        String usuario = seguridad.getCallerPrincipal().getName();
```

```
        String operacion = ctx.getMethod().getName();
        try {
            Object resultado = ctx.proceed();
            registro.exito(usuario, operacion, ctx.getParameters());
            return resultado;
        } catch (Exception e) {
            registro.fallo(usuario, operacion, e.getClass().getSimpleName());
            throw e;
        }
    }
}
```

4. Patrones GoF

4.1 Singleton (@Singleton EJB)

Aspecto	Detalle
Problema en MediConecta	El catálogo de prestaciones y de obras sociales se consulta en cada reserva y casi no cambia. Ir a la base cada vez es costo innecesario.
Solución	Un EJB @Singleton con @Startup carga el catálogo al iniciar y lo recarga de madrugada. El contenedor garantiza una sola instancia y maneja la concurrencia con @Lock.
Participantes (rol del patrón: clase)	Singleton: CatalogoPrestaciones. Clientes: ValidadorReserva, CalculoCopago.
Consecuencias y alternativa descartada	Lecturas concurrentes sin bloqueo (LockType.READ) y recarga exclusiva (WRITE). Se descartó el Singleton clásico con constructor privado: en un servidor de aplicaciones no garantiza una instancia única por aplicación y queda fuera del ciclo de vida del contenedor.

```
@Singleton @Startup @Lock(LockType.READ)
public class CatalogoPrestaciones {
    private Map<String, Prestacion> porCodigo;

    @PostConstruct
    void cargar() { porCodigo = dao.todas().stream()
                        .collect(toMap(Prestacion::getCodigo, p -> p)); }

    public Prestacion buscar(String codigo) { return porCodigo.get(codigo); }

    @Lock(LockType.WRITE) @Schedule(hour = "3")
    void recargar() { cargar(); }
}
```

4.2 Builder

Aspecto	Detalle
Problema en MediConecta	El evento del turno tiene muchos campos, varios opcionales (fechaHoraAnterior solo en reprogramaciones) y otros que siempre se generan igual (idEvento, timestamp). Un constructor con diez parámetros es ilegible y propenso a errores.
Solución	Un builder interno arma el evento paso a paso y en build() valida lo obligatorio y genera el idEvento y la marca de tiempo.
Participantes (rol del patrón: clase)	Builder: EventoTurno.Builder. Producto: EventoTurno. Director: TurnoService.
Consecuencias y alternativa descartada	Imposible publicar un evento sin idEvento, del que depende la idempotencia. Se descartó un constructor telescópico.

```
EventoTurno evento = EventoTurno.builder()
        .tipo(TipoEvento.TURNO_REPROGRAMADO)
        .turno(turno)
        .fechaHoraAnterior(horarioAnterior)   // solo en reprogramaciones
        .build();                              // genera idEvento y timestamp
```

4.3 Template Method con Factory Method

Aspecto	Detalle
Problema en MediConecta	Notificar por email y por SMS sigue los mismos pasos (armar el mensaje, verificar que el paciente acepte ese canal, enviar, registrar), pero el formato y el envío cambian según el canal.
Solución	Una clase abstracta fija el algoritmo en un método final (Template Method). El paso de creación del mensaje es un método abstracto que cada subclase redefine (Factory Method), como describe GoF al combinar ambos patrones.
Participantes (rol del patrón: clase)	Clase abstracta: ProcesadorNotificacion. Clases concretas: ProcesadorEmail, ProcesadorSms. Producto: Mensaje (MensajeEmail, MensajeSms).
Consecuencias y alternativa descartada	Sumar un canal es agregar una subclase sin tocar el flujo. Se descartó Strategy para el canal porque lo que varía no es un algoritmo aislado sino pasos de un proceso con orden fijo.

```
public abstract class ProcesadorNotificacion {
    public final void procesar(EventoTurno ev) {           // Template Method
        if (!ev.paciente().aceptaCanal(canal())) return;
        Mensaje mensaje = crearMensaje(ev);                 // Factory Method
        enviar(mensaje);
        auditoria.registrarEnvio(ev.idEvento(), canal());
    }
    protected abstract Mensaje crearMensaje(EventoTurno ev);
    protected abstract void enviar(Mensaje mensaje);
    protected abstract Canal canal();
}
```

4.4 Adapter / Gateway

Aspecto	Detalle
Problema en MediConecta	El servicio de cobertura de la obra social es un sistema legado SOAP con su propio formato. Si el dominio usara directamente las clases generadas por wsimport, cualquier cambio del legado se propagaría a todo el sistema.
Solución	El dominio define su propia interfaz (Gateway) y un adaptador la implementa traduciendo hacia y desde el cliente JAX-WS.
Participantes (rol del patrón: clase)	Target: CoberturaGateway. Adapter: CoberturaSoapAdapter. Adaptee: CoberturaService (port JAX-WS del legado, simulado con un mock JAX-WS).
Consecuencias y alternativa descartada	El cambio del legado toca una sola clase, y en los tests se reemplaza el gateway por uno falso. Se descartó llamar al port desde TurnoService.

4.5 Chain of Responsibility

Aspecto	Detalle
Problema en MediConecta	Antes de reservar hay que verificar afiliado activo, horario libre, cobertura vigente y que el paciente no tenga otro turno superpuesto. Las reglas cambian (una obra social puede exigir algo más) y un método con todos los if crece sin control.
Solución	Cada regla es un eslabón. La cadena se arma en un solo lugar y la solicitud recorre los eslabones; el primero que falla corta con una excepción de negocio.
Participantes (rol del patrón: clase)	Handler: ValidadorReserva. Handlers concretos: AfiliadoActivo, HorarioLibre, CoberturaVigente, SinSuperposicion. Cliente: TurnoService.
Consecuencias y alternativa descartada	Agregar o reordenar reglas no toca el servicio de reserva. Costo: más clases chicas. Se descartó un único método validador.

```
public abstract class ValidadorReserva {
    private ValidadorReserva siguiente;
    public ValidadorReserva enlazar(ValidadorReserva s) { siguiente = s; return s; }
    public void validar(SolicitudTurno solicitud) {
        verificar(solicitud);                       // lanza ReservaInvalidaException
        if (siguiente != null) siguiente.validar(solicitud);
    }
    protected abstract void verificar(SolicitudTurno solicitud);
}
```

```
ValidadorReserva cadena = new AfiliadoActivo();
cadena.enlazar(new HorarioLibre())
      .enlazar(new CoberturaVigente())
      .enlazar(new SinSuperposicion());
```

4.6 State

Aspecto	Detalle
Problema en MediConecta	El turno pasa por pendiente de autorización, confirmado, reprogramado, cancelado, atendido y ausente, y cada estado admite transiciones distintas (no se cancela un turno atendido). Con switch sobre un enum, cada operación repite la tabla de transiciones.
Solución	Cada estado es una clase que implementa las operaciones permitidas y rechaza las inválidas. La entidad persiste un enum y la clase de estado se resuelve a partir de él.
Participantes (rol del patrón: clase)	Context: Turno. State: EstadoTurno. Estados concretos: PendienteAutorizacion, Confirmado, Reprogramado, Cancelado, Atendido, Ausente.
Consecuencias y alternativa descartada	Las transiciones inválidas son imposibles por diseño y cada estado se testea por separado. Se descartó el switch sobre el enum.

```
public class Confirmado implements EstadoTurno {
    public EstadoTurno cancelar(Turno t)     { return new Cancelado(); }
    public EstadoTurno reprogramar(Turno t, LocalDateTime nueva) {
        t.setFechaHora(nueva);
        return new Reprogramado();
    }
    public EstadoTurno marcarAtendido(Turno t) { return new Atendido(); }
}

public class Atendido implements EstadoTurno {
    public EstadoTurno cancelar(Turno t) {
        throw new TransicionInvalidaException("Un turno atendido no se cancela");
    }
}
```

4.7 Strategy

Aspecto	Detalle
Problema en MediConecta	El copago se calcula distinto según la obra social (porcentaje, monto fijo, exento por plan). Un if por obra social en el servicio obliga a modificarlo cada vez que se suma una.
Solución	Cada regla de cálculo es una estrategia. CDI descubre todas las implementaciones y se elige la que aplica a la obra social del afiliado.
Participantes (rol del patrón: clase)	Strategy: CalculoCopago. Estrategias concretas: CopagoPorcentual, CopagoMontoFijo, CopagoExento. Context: TurnoService.
Consecuencias y alternativa descartada	Sumar una obra social es agregar una clase. En la entrega final se reutiliza el patrón para la política de asignación de la lista de espera (orden de llegada o prioridad clínica).

```
@Inject @Any Instance<CalculoCopago> estrategias;

CalculoCopago elegir(String obraSocial) {
    return estrategias.stream()
                      .filter(e -> e.aplica(obraSocial))
                      .findFirst()
                      .orElse(copagoPorDefecto);
}
```

4.8 Decorator (CDI)

Aspecto	Detalle
Problema en MediConecta	La ficha del paciente la consultan profesionales y administrativos, pero solo el profesional debe ver los datos clínicos. Meter esa regla dentro de PacienteService mezcla negocio con control de acceso.
Solución	Un decorador CDI envuelve el servicio original y enmascara los datos clínicos cuando quien llama no tiene el rol PROFESIONAL.
Participantes (rol del patrón: clase)	Component: PacienteService. Concrete component: PacienteServiceImpl. Decorator: PacienteEnmascarado.
Consecuencias y alternativa descartada	El servicio no cambia y la regla de privacidad queda en un solo lugar. Se descartó resolverlo con dos métodos distintos en el servicio.

```
@Decorator @Priority(Interceptor.Priority.APPLICATION)
public abstract class PacienteEnmascarado implements PacienteService {
    @Inject @Delegate @Any PacienteService delegado;
    @Inject SecurityContext seguridad;

    public FichaPacienteDTO ficha(long idPaciente) {
        FichaPacienteDTO ficha = delegado.ficha(idPaciente);
        return seguridad.isCallerInRole("PROFESIONAL") ? ficha : ficha.sinDatosClinicos();
    }
}
```

5. Patrones de integración y mensajería

Nombres según Hohpe y Woolf, Enterprise Integration Patterns.

Patrón	Aplicación en MediConecta	Por qué
Publish-Subscribe Channel	topico.turnos con Notificaciones, Auditoría y Facturación	Un evento con varios interesados que hacen cosas distintas, sin que el productor los conozca
Point-to-Point Channel	cola.autorizaciones y cola.revalidacion	Tareas que deben procesarse exactamente una vez
Event-Driven Consumer	Message Driven Beans	El contenedor despierta al consumidor cuando llega el mensaje, sin polling
Competing Consumers	Dos instancias de AutorizacionConsumer sobre la misma cola	Absorber los picos de principio de mes sin duplicar trabajo
Selective Consumer	Facturación filtra tipoEvento IN (TURNO_CONFIRMADO, TURNO_CANCELADO)	Un solo tópico para todos los eventos del turno, cada suscriptor recibe lo que le sirve
Durable Subscriber	Suscripciones con nombre de Facturación y Auditoría	Si el suscriptor está caído, el broker le retiene los mensajes
Guaranteed Delivery	Mensajes persistentes (modo de entrega por defecto)	Un reinicio del broker no pierde eventos
Idempotent Receiver	Insert en evento_procesado con clave única sobre idEvento	El broker puede entregar más de una vez y no se puede facturar dos veces
Transactional Client	Envío JMS dentro de la transacción JTA del alta del turno	Evita publicar el evento de un turno cuya transacción hizo rollback
Dead Letter Channel	Tras 3 intentos fallidos, el mensaje va a jms.queue.DLQ	Lo que no se puede procesar no se pierde ni bloquea la cola
Message Expiration	TTL de 24 horas en las solicitudes de autorización	La cola no crece sin límite; lo vencido va a la cola de expirados

5.1 Transactional Client: publicar solo si el turno se guardó

Si el turno se persiste y el evento se publica en dos pasos independientes, una falla entre ambos deja un evento publicado de un turno que no existe, o un turno sin su evento. En WildFly, el JMSContext inyectado usa la fábrica de conexiones pooled y XA del servidor, así que el envío se enlista en la misma transacción JTA del EJB: el mensaje sale recién con el commit, y si hay rollback no sale.

```
@Stateless
public class TurnoService {
    @PersistenceContext EntityManager em;
    @Inject JMSContext jms;                                  // XA, enlistado en la JTA
    @Resource(lookup = "java:/jms/topic/turnos") Topic topico;

    public Turno confirmar(Turno turno) {                     // REQUIRED por defecto
        em.persist(turno);
        jms.createProducer()
           .setProperty("tipoEvento", "TURNO_CONFIRMADO")
           .send(topico, EventoTurno.de(turno).toJson());
        return turno;                                         // BD y envío: commit juntos
    }
}
```

5.2 Durable Subscriber, Selective Consumer e Idempotent Receiver en el MDB

```
@MessageDriven(activationConfig = {
  @ActivationConfigProperty(propertyName = "destinationLookup",
                            propertyValue = "java:/jms/topic/turnos"),
  @ActivationConfigProperty(propertyName = "destinationType", propertyValue = "jakarta.jms.Topic"),
  @ActivationConfigProperty(propertyName = "subscriptionDurability", propertyValue = "Durable"),
  @ActivationConfigProperty(propertyName = "subscriptionName", propertyValue = "facturacion"),
  @ActivationConfigProperty(propertyName = "clientId", propertyValue = "mediconecta-facturacion"),
  @ActivationConfigProperty(propertyName = "messageSelector",
                            propertyValue = "tipoEvento IN ('TURNO_CONFIRMADO','TURNO_CANCELADO')")
})
@RunAs("SISTEMA")
public class FacturacionSubscriber implements MessageListener {
    public void onMessage(Message m) {
        EventoTurno ev = EventoTurno.desdeJson(m.getBody(String.class));
        try {
            procesados.registrar(ev.idEvento());   // clave única: falla si ya se procesó
        } catch (EventoDuplicadoException dup) {
            return;                                 // duplicado: se descarta sin efectos
        }
        facturacion.procesar(ev);
    }
}
```

La idempotencia se apoya en la restricción de unicidad de la base y no en un SELECT previo, porque dos instancias concurrentes podrían consultar a la vez y ambas encontrar que el evento no existe.

5.3 Dead Letter Channel y Message Expiration

```
# jboss-cli: reintentos y cola de mensajes fallidos para la cola de autorizaciones
cd /subsystem=messaging-activemq/server=default
./address-setting=jms.queue.autorizaciones:add(max-delivery-attempts=3, redelivery-delay=10000)
cd address-setting=jms.queue.autorizaciones
:write-attribute(name=dead-letter-address, value=jms.queue.DLQ)
:write-attribute(name=expiry-address, value=jms.queue.ExpiryQueue)

// En el productor: las solicitudes de autorización vencen a las 24 horas
jms.createProducer().setTimeToLive(86_400_000).send(colaAutorizaciones, solicitud);
```

6. Patrones de resiliencia ante el sistema legado

Responden al desafío de la Clase 9: qué hace el sistema si el SOAP del legado no responde en cinco segundos. WildFly incluye MicroProfile Fault Tolerance (configuración standalone-microprofile.xml), que implementa estos patrones con anotaciones.

Patrón	Comportamiento
Timeout	Corta la llamada al legado a los 5 segundos

Patrón	Comportamiento
Retry	Un reintento con 500 ms de espera. Es seguro porque validar cobertura es una consulta sin efectos
Circuit Breaker	Tras 5 fallas seguidas deja de llamar al legado durante 2 minutos
Fallback con Cache-Aside	Responde con la última validación vigente en cache; si no hay, devuelve cobertura pendiente, el turno queda en COBERTURA_PENDIENTE y se encola en cola.revalidacion

```
@ApplicationScoped
public class CoberturaSoapAdapter implements CoberturaGateway {

    @Timeout(5000)
    @Retry(maxRetries = 1, delay = 500)
    @CircuitBreaker(requestVolumeThreshold = 5, failureRatio = 1.0, delay = 120_000)
    @Fallback(fallbackMethod = "desdeCache")
    public ResultadoCobertura validar(Afiliado afiliado, Prestacion prestacion) {
        ValidacionResponse r = puerto.validarCobertura(afiliado.getObraSocial(),
                afiliado.getNroAfiliado(), prestacion.getCodigo(), ...);
        cache.guardar(afiliado, prestacion, r);
        return ResultadoCobertura.desde(r);
    }

    ResultadoCobertura desdeCache(Afiliado afiliado, Prestacion prestacion) {
        return cache.vigente(afiliado, prestacion)
                    .orElseGet(ResultadoCobertura::pendiente);
    }
}
```

Si la cátedra no admite MicroProfile, el mismo comportamiento se implementa a mano: un contador de fallas en un @Singleton para el breaker y un bloque try/catch para el reintento y el fallback.

7. Patrones evaluados y descartados

Patrón	Motivo del descarte
Observer	Lo provee JMS con el tópico. Implementarlo a mano duplicaría lo que ya resuelve el broker
Service Locator y Business Delegate	Reemplazados por la inyección de CDI y @EJB. No hay clientes remotos de EJB
Command y Memento	Solo aportan si hay que deshacer acciones (revertir una cancelación), y no es un requerimiento. El historial de cambios ya queda en la auditoría
Abstract Factory	Se integra una sola obra social. Se reevalúa si se suman varias con familias de componentes propias
Correlation Identifier	El consumidor de autorizaciones resuelve contra el web service de forma sincrónica, no hay mensaje de respuesta que correlacionar
Claim Check	Publicar solo identificadores y que cada consumidor consulte los datos reduce la exposición de datos personales en el broker, pero agrega una consulta por evento. Se mantiene el evento con datos mínimos de contacto y sin datos clínicos
Visitor, Prototype, Flyweight, Composite	No hay en el dominio un problema concreto que resuelvan

8. Diseño de seguridad

8.1 Marco

MediConecta trata datos de salud. La Ley 25.326 de Protección de Datos Personales los define como datos sensibles (art. 2) y regula su tratamiento (art. 8), y la Ley 26.529 de Derechos del Paciente establece la confidencialidad de la historia clínica. Las decisiones de esta sección responden a ese marco: control de acceso por rol y por titularidad, cifrado en tránsito y en reposo, minimización de datos en los mensajes y trazabilidad de los accesos.

8.2 Roles y matriz de permisos

Los casilleros con (*) no se resuelven solo con el rol: además hay que verificar que el recurso pertenezca a quien llama (seguridad programática, punto 8.4).

Operación	PACIENTE	PROFESIONAL	ADMINISTRATIVO	AUDITOR	SISTEMA
Reservar turno	Sí, propio (*)	No	Sí	No	No
Cancelar turno	Propio (*)	De su agenda (*)	Sí	No	No
Consultar turnos	Propios (*)	De su agenda (*)	Todos	No	No
Registrar atención y diagnóstico	No	Sus pacientes (*)	No	No	No
Ver ficha clínica	Propia, resumida (*)	Pacientes atendidos (*)	Solo datos administrativos	No	No
Gestionar agendas	No	Propia (*)	Todas	No	No
Consultar auditoría	No	No	No	Sí	No
Emitir o anular facturación	No	No	No	No	Sí, desde el MDB

8.3 Autenticación con Jakarta Security

Se usa la Jakarta Security API, que es el estándar de la plataforma. Para las entregas alcanza con autenticación BASIC sobre TLS y un identity store sobre la base, con contraseñas hasheadas con PBKDF2 (incluido en la especificación, sin librerías externas). En la entrega final se evoluciona a tokens JWT con MicroProfile JWT para la API REST.

```
@BasicAuthenticationMechanismDefinition(realmName = "mediconecta")
@DatabaseIdentityStoreDefinition(
    dataSourceLookup = "java:jboss/datasources/MediConectaDS",
    callerQuery      = "select password from usuario where username = ?",
    groupsQuery      = "select rol from usuario_rol where username = ?",
    hashAlgorithm    = Pbkdf2PasswordHash.class)
@ApplicationScoped
public class ConfiguracionSeguridad {}
```

En WildFly hay que habilitar Jakarta Security en el dominio de seguridad de la aplicación:

```
cd /subsystem=undertow/application-security-domain=other
:write-attribute(name=integrated-jaspi, value=false)
```

8.4 Autorización declarativa y programática

La declarativa (@DeclareRoles, @RolesAllowed, @PermitAll, @DenyAll en TurnoFacade, ver 3.2) decide si un rol puede ejecutar una operación. No alcanza para la regla "un paciente solo opera sus propios turnos": un paciente autenticado podría cancelar el turno de otro cambiando el id en la URL (referencia directa insegura a objetos, IDOR). Eso se resuelve con seguridad programática:

```
@Resource SessionContext ctx;

public void cancelar(long idTurno) {
    Turno turno = dao.buscar(idTurno);
    String usuario = ctx.getCallerPrincipal().getName();
    if (ctx.isCallerInRole("PACIENTE") && !turno.perteneceA(usuario)) {
        throw new EJBAccessException("El turno no pertenece al usuario");
    }
    turno.cancelar();
}
```

8.5 Seguridad en la mensajería

- **Identidad de los MDB.** Un Message Driven Bean no tiene usuario que lo invoque. Si llama a un EJB protegido con @RolesAllowed, la llamada se rechaza. Por eso los consumidores se ejecutan con @RunAs("SISTEMA"), un rol técnico que solo tiene permiso para las operaciones que el procesamiento asincrónico necesita (ver 5.2).

- **Permisos en el broker.** Se deshabilita el acceso del usuario guest que trae Artemis por defecto y se definen roles por destino: solo la aplicación puede publicar en el tópico de turnos y solo los consumidores pueden leer las colas.
- **Minimización de datos.** El evento del tópico no lleva diagnóstico ni datos clínicos. Facturación recibe cobertura y código de prestación; Notificaciones, solo los datos de contacto.

```
cd /subsystem=messaging-activemq/server=default
./security-setting=jms.topic.turnos:add
cd security-setting=jms.topic.turnos
./role=aplicacion:add(send=true, consume=false)
./role=consumidor:add(send=false, consume=true, create-durable-queue=true)
```

El nombre exacto de cada dirección se verifica en la consola de administración de WildFly antes de aplicar los comandos.

8.6 Protección de los datos

- **En tránsito:** HTTPS en Undertow para REST y SOAP. Para la demo alcanza con un certificado autofirmado.
- **En reposo:** el diagnóstico y el número de afiliado se cifran a nivel de columna con un AttributeConverter de JPA (AES-GCM). La clave no está en el código: se guarda en el credential store de Elytron.
- **Contraseñas:** PBKDF2 con sal, nunca en texto plano.
- **Servicio SOAP:** WS-Security con UsernameToken sobre TLS (WildFly usa Apache CXF, que lo soporta por configuración).

```
@Converter
public class CifradoConverter implements AttributeConverter<String, String> {
    public String convertToDatabaseColumn(String valor)  { return cifrador.cifrar(valor); }
    public String convertToEntityAttribute(String valor) { return cifrador.descifrar(valor); }
}

@Convert(converter = CifradoConverter.class)
private String diagnostico;
```

8.7 Validación y manejo de errores

- Bean Validation en los DTO de entrada (@NotNull, @Pattern sobre el número de afiliado, @Future sobre la fecha del turno).
- Consultas JPA siempre con parámetros, nunca concatenando texto en el JPQL.
- Un ExceptionMapper devuelve mensajes genéricos: ninguna respuesta REST expone una traza de excepción ni nombres de tablas. Los SOAP Fault llevan un código de negocio, no el detalle interno.

8.8 Riesgos OWASP Top 10 y su mitigación

Riesgo	Mitigación en MediConecta
Control de acceso roto	@RolesAllowed según la matriz y verificación de titularidad contra IDOR (8.4)
Fallas criptográficas	TLS, PBKDF2 y cifrado de columnas sensibles (8.6)
Inyección	JPA con parámetros y Bean Validation (8.7)
Diseño inseguro	Matriz de permisos y minimización de datos definidas en el diseño, antes del código
Configuración de seguridad incorrecta	Usuario guest de Artemis deshabilitado, consola de administración no expuesta, sin trazas en las respuestas
Componentes vulnerables	OWASP Dependency-Check en el build de Maven, con el reporte en el repositorio
Fallas de autenticación	Identity store con hash, expiración de sesión y de token
Integridad de datos	Solo la aplicación puede publicar en el tópico (8.5)
Fallas de registro y monitoreo	Interceptor de auditoría sobre toda operación con datos clínicos (3.3)
Falsificación de peticiones del lado del servidor	El único destino saliente es el endpoint del legado, fijo en configuración y no armado con datos del usuario

8.9 Incorporación por entrega

Entrega	Seguridad que se incorpora
E3 (19/10), mensajería	Guest deshabilitado y roles por destino en Artemis, @RunAs en los MDB, evento sin datos clínicos
E4 (09/11), integración	Jakarta Security con PBKDF2, matriz de permisos con @RolesAllowed, control de titularidad, Bean Validation, ExceptionMapper, HTTPS, WS-Security en el SOAP
Final (30/11)	JWT con MicroProfile JWT, cifrado de columnas, decorador de enmascarado, auditoría completa, tabla OWASP y Dependency-Check

9. Trazabilidad con el checklist de la cátedra

Ítem del checklist	Dónde se cumple
Servicio SOAP con WSDL	CoberturaService: mock JAX-WS del legado de la obra social, consumido por CoberturaSoapAdapter
Servicio REST	TurnoResource (JAX-RS) sobre TurnoFacade
JMS: cola	cola.autorizaciones y cola.revalidacion
JMS: tópico	topico.turnos con tres suscriptores durables
EJB stateless	TurnoFacade, TurnoService
EJB stateful	ReservaEnCurso: asistente de reserva paso a paso (especialidad, profesional, horario) que conserva la selección entre pantallas
Tres o más patrones	22 patrones aplicados (sección 2), con 7 evaluados y descartados (sección 7)
Seguridad declarativa	@DeclareRoles y @RolesAllowed en TurnoFacade según la matriz de permisos (8.2)
Transacciones	CMT con REQUIRED en TurnoFacade y TurnoService; envío JMS dentro de la transacción JTA (5.1)